Project Report Template

Advanced Analytics and Applications

Team 05

Authors:
George Sadik (111111111)
Kushale Fernando (222222222)
Claus Hack (333333333)
Martin Nikoljacic (333333333)
Contact: student@smail.uni-koeln.de

Supervisor: Univ.-Prof. Dr. Wolfgang Ketter
Co-Supervisor: Janik Muires

Chair of Information Systems for Sustainable Society
Faculty of Management, Economics and Social Sciences
University of Cologne

2026-04-01

Table of contents

List of Figures

1 Introduction

This report serves as a template for your team project. It is built using Quarto, which allows you to integrate your Python or R analysis directly into your writing.

By using this template, you ensure that your formatting (margins, line spacing, and fonts) matches the required standards for submission. The left margin is set to 4cm to allow for comments, while the other margins are 2.5cm. Line spacing is set to 1.5, and the main font is Times New Roman.

2 Preprocessing

2.1 Support Vector Machines (SVM)

The SVR model has four dimensions of testing: two spatial layers (448 census tracts or 77 community areas), target encoding (with or without), the target transformation (normal or log1p) and three temporal resolutions (1h, 4h, 1d). Following the assignment, we start without a kernel using a LinearSVR baseline and gradually add complexity through RBF and polynomial kernels. All 24 combinations are tuned across these three Kernels. We compute all 24 combinations once and analyze them in four controlled comparisons, where each phase varies exactly one factor while all others stay fixed. This isolates the effect of every design decision and still yields the overall best SVR as a benchmark against the neural network. The full feature set can be found in Appendix X.

One technical limitation shapes the whole setup. Kernel SVR scales roughly quadratically with the number of samples, so training on the full dataset (up to several million rows) is not feasible. Grid search therefore fits on a 20,000-row subsample and the final model is refit on 50,000 rows. This cap is a property of the method itself and will matter again when we compare against the neural network.

Phase 1 compares the two spatial layers at 1h resolution with target encoding and the untransformed target. Phase 2 compares with and without target encoding at 1h across both spatial layers. Target encoding adds one column containing the mean trip count per spatial unit, computed on the training set only, so the model learns whether a location is generally busy or quiet.[1] Phase 3 compares the untransformed target against log1p on community areas. Trip counts are strongly right-skewed, so a few high-demand baskets dominate the squared error, and log1p compresses these extremes. To avoid selecting a kernel that looks strong on the log scale but fails after back-transformation, the grid search uses a custom RMSE scorer that applies expm1 before computing the error on the original scale. The final phase compares the three temporal resolutions of 1h, 4h and 1d on community areas with target encoding. Before evaluating, we verified that the chronological test set is representative of the training data; the target distributions of both splits are compared in Figure 1.

Table 1: SVR results across four controlled comparison phases

Phase	Configuration	Kernel	R^2	rMAE
1: spatial	census_tract 1h	Poly	0.66	1.17
1: spatial	community_area 1h	Poly	0.83	0.44
2: target enc.	census_tract 1h + TE	Poly	0.66	1.17
2: target enc.	census_tract 1h (no TE)	Poly	0.22	1.58

[1] Dummy encoding was rejected because it would add up to 448 sparse columns, which is computationally expensive given the quadratic scaling of kernel SVR.

Phase	Configuration	Kernel	R^2	rMAE
2: target enc.	community_area 1h + TE	Poly	0.83	0.44
2: target enc.	community_area 1h (no TE)	Poly	0.83	0.68
3: transform	community_area 1h normal	Poly	0.83	0.44
3: transform	community_area 1h log1p	RBF	0.83	0.40
4: temporal	community_area 1h	Poly	0.83	0.44
4: temporal	community_area 4h	Poly	0.88	0.39
4: temporal	community_area 1d	Poly	0.93	0.23

Community areas clearly outperform census tracts (R^2 0.83 against 0.66, rMAE 0.44 against 1.17).[2] Target encoding is what keeps the census tract layer usable at all. Without it, R^2 collapses from 0.66 to 0.22. On community areas the effect on R^2 is marginal (0.83 against 0.83), but the normalized error still improves from 0.68 to 0.44. The full results across all 24 configurations are shown in Table 3. Log1p behaves similarly at 1h, where R^2 stays the same and only the error improves, but at 4h the transform lifts R^2 from 0.88 to 0.89 and reduces rMAE from 0.39 to 0.32. Wider time windows improve every metric (R^2 0.83 to 0.88 to 0.93) because aggregation averages out short-term noise. Across the grid, the polynomial kernel performs best on the untransformed target, while the RBF kernel wins whenever log1p is applied. A visual comparison of all three kernel families can be found in Figure 2.

The best raw performance is community areas at daily resolution without target encoding and the RBF kernel (R^2 0.93, rMAE 0.23). Since a daily total per neighborhood is too coarse for actual fleet coordination, we select community areas at 4h with target encoding, log1p and an RBF kernel (R^2 0.89, rMAE 0.32) as the final SVR benchmark against the neural network. The full discussion of which resolution the client should deploy follows in the final recommendation.

This leaves the model with three notable shortfalls. The scaling cap described above means the SVR never sees more than a fraction of the available data. The log1p transform, needed to handle the skewed target, compresses high values and therefore tends to under-predict extreme peaks, which are exactly the baskets that matter most on event days. And at fine spatial resolution the model depends entirely on manual feature engineering. Without target encoding, census tract performance collapses. For a follow-up project, the clearest improvement levers target the scaling cap directly. Kernel approximations such as Nyström would reduce the quadratic cost to a linear one and let the SVR train on the full dataset, while gradient-boosted trees such as LightGBM handle large sample sizes and non-linear interactions natively and would replace the kernel machinery altogether. The error distribution per community area for the recommended configuration can be found in Figure 3.

2.2 Neural Networks

To ensure a fair comparison with the Support Vector Machine (SVM), the neural network experiments use the same spatial and temporal bins, the same train/validation/test split methodology and the same metrics.

The neural network is implemented as a feed-forward network. First, the data is preprocessed by scaling the numerical features and converting the spatial unit into numerical labels that are passed through an embedding. A baseline experiment on the 1-hour community area dataset is then used to determine the best number of hidden layers. The training loss curves for all experiments are shown in Figure 6. The network is trained on an L1 loss, which weights

[2] Due to different scales across temporal and spatial resolutions, the normalized MAE (MAE divided by mean demand) is used to enable a valid comparison.

large errors linearly and is therefore robust to the strongly right-skewed target without requiring a transformation. Grid search is performed separately for each experiment to find the best remaining hyperparameters, including the embedding size, hidden layer width, learning rate, dropout, weight decay, and batch size, consistent with the per-configuration tuning used for the SVM. The network reaches an R^2 of 0.92 on community areas at 1h and remains at 0.85 on census tracts. The full results per configuration are shown in Table 2. Detailed spatial and temporal resolution comparisons are visualized in Figure 4 and Figure 5. Compared with community areas, census tracts introduce substantially more spatial categories and many more low-demand or zero-demand observations, creating a much sparser learning problem. Although the embedding layer helps the network learn spatial relationships more effectively than traditional categorical encoding, the finer spatial resolution still makes it more difficult for the model to generalize. Predicted-versus-actual scatter plots for all configurations can be found in Figure 7.

The model nevertheless has limitations. The largest is computational power. Due to limited hardware resources, the depth search was performed only once on the baseline experiment rather than independently for every configuration, and hyperparameter tuning had to run on reduced training subsets. For the same reason only the four configurations required for the direct comparison with the SVM were run, rather than the full grid. The selected configurations are therefore unlikely to represent the absolute best-performing models. In addition, several choices such as the number of epochs, the early-stopping threshold, the loss function and the optimizer were selected through trial and error. Future work could address exactly these points. An independent depth search per experiment, larger search spaces with more computational resources, and a systematic comparison of loss functions and optimizers instead of trial and error.

2.3 Model Comparison and Recommendation

Both models are evaluated on identical held-out test sets across four configurations, shown in Table 2. As described in the SVM section, the SVR had to be fitted on subsamples of 20,000 and 50,000 rows, while the network could be trained on the full data of 1.1 million rows for community areas at 1h and 6.8 million for census tracts. Part of the gap between the two models therefore reflects how much data each could use. We treat this as a property of the methods. Being able to use a dataset of this size is one of the advantages of the neural network as discussed in the lecture.

Table 2: Comparison of SVR and FFNN across matched spatio-temporal configurations

Configuration	SVR R^2	FFNN R^2	SVR rMAE	FFNN rMAE
Community area, 1h	0.83	0.92	0.44	0.20
Census tract, 1h	0.66	0.85	1.17	0.23
Community area, 4h	0.89	0.93	0.32	0.17
Community area, 1d	0.93	0.93	0.23	0.15

On R^2 the two models perform similarly as the time window widens. At 1d both reach 0.93. The difference in performance can be seen with more detailed resolutions. At 1h bins in community areas the SVR reaches 0.83 against 0.92, and on census tracts it falls to 0.66 while the network holds 0.85. The SVR only manages to keep up with the network when the baskets are aggregated into larger ones. The network stays more stable even in finer resolutions. The normalized MAE says the same. The network shows a lower error in every configuration, including the daily setting where R^2 is tied (0.15 against 0.23).

Neither model is simpler to build. The SVR needed target encoding, log1p and a custom scorer. The network replaces the first with embeddings and avoids the other two by training on an L1 loss, which is robust to the skewed target, but requires architecture search, dropout and learning-rate tuning instead. What separates them is scale. The SVR cannot use the full dataset, and that shows at the fine resolutions where the extra data matters most.

A more direct test is whether either model beats an operator working without one. We assume a fleet operator would rely on historical averages per community area, 4h basket and weekday/weekend split, essentially saying "on a Tuesday between 8 and 12 in community area 10 we expect 8.2 trips". This rule of thumb reaches an rMAE of 0.31 on the test set. The SVR does not beat it (0.32), most likely because it only sees a fraction of the training data, while the network reaches 0.17, an improvement of roughly 45 percent. For the SVR the modelling effort does not pay off against a simple historical average while for the network it clearly does. The network is therefore the model we would put into production. In practical terms, the network predicts demand within five trips of the actual count in roughly 83% of all 4h test windows (see Table 5).

That leaves the question of which resolution to run it at. We recommend community areas in four-hour windows (R^2 0.93, rMAE 0.17). Daily forecasts score marginally better (R^2 0.93, rMAE 0.15) but are too coarse for a fleet operator to work with, while hourly forecasts add noise without adding planning value. Census tracts were considered for their finer spatial detail but rejected. Fewer than half of the records contain census tract information (see Chapter 1), and many tracts generate fewer than one trip per window, which is too granular to allocate vehicles against. Community areas are small enough to act on. The average community area in Chicago covers roughly three square miles,[3] which a taxi can cross in under fifteen minutes.

3 Smart Charging Using Reinforcement Learning

4 Discussion & Outlook

5 Citations and References

You can easily cite your sources using BibLaTeX. For example, you can cite the Quarto documentation (Team, 2024) or a textbook like "R for Data Science" (Wickham et al., 2023). The bibliography will be automatically generated at the end of the document.

6 Conclusion

The conclusion should summarize your findings and suggest future work.

[3] Chicago comprises 77 community areas across roughly 227 square miles (U.S. Census Bureau).

7 Appendix

7.1 SVR Results

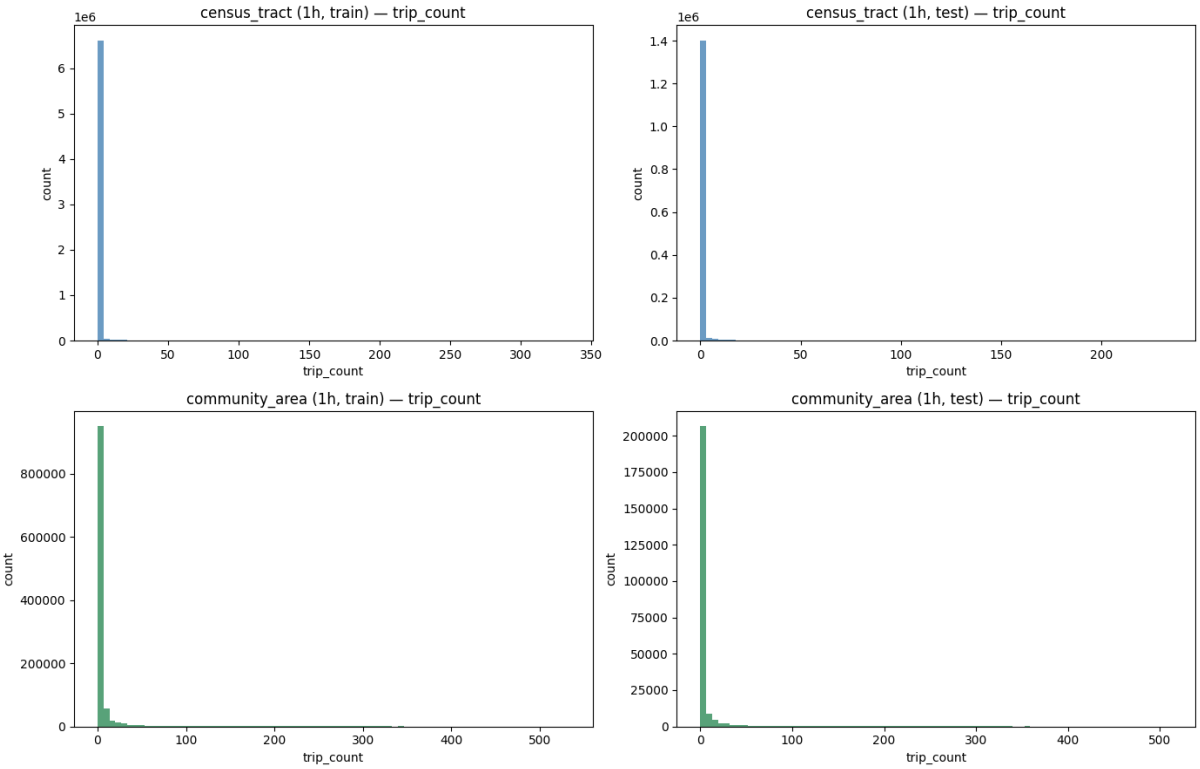

Figure 1: Distribution of trip counts in training and test set (community areas, 1h).

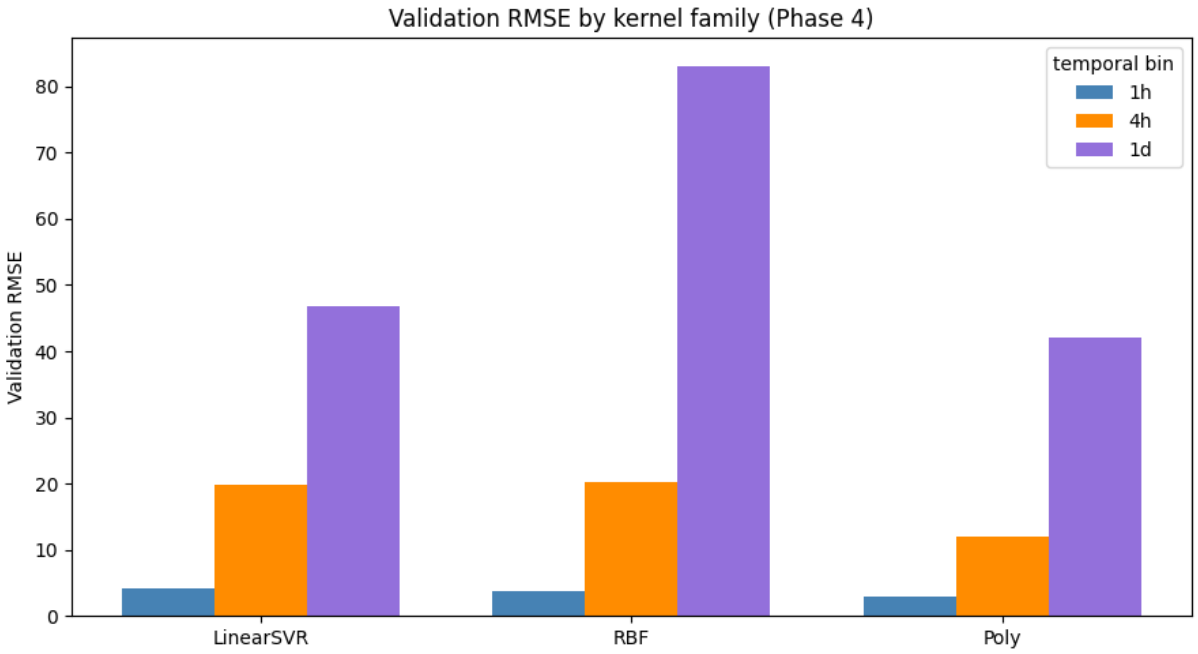

Figure 2: Validation RMSE by kernel family across all configurations.

Table 3: Full regression metrics across all 24 SVR configurations.

Config	Kernel	R2	RMAE
community_area_1d	Poly	0.9332	0.2305
community_area_1d (no TE)	Poly	0.9326	0.2529
community_area_1d + log1p	Poly	0.9299	0.2313
community_area_4h + log1p (no TE)	RBF	0.9122	0.2977
community_area_4h + log1p	RBF	0.8929	0.3207
census_tract_1d	Poly	0.8891	0.4049
community_area_4h	Poly	0.8795	0.3901
community_area_4h (no TE)	Poly	0.8656	0.4319
community_area_1h	Poly	0.8343	0.4356
community_area_1h (no TE)	Poly	0.8287	0.6815
community_area_1h + log1p	RBF	0.8286	0.4005
community_area_1h + log1p (no TE)	RBF	0.8285	0.4083
census_tract_1d + log1p	RBF	0.8212	0.4544
census_tract_4h	Poly	0.7384	0.7763
community_area_1d + log1p (no TE)	Poly	0.6641	0.3994
census_tract_1h	Poly	0.6562	1.1729
census_tract_4h + log1p	RBF	0.6389	0.6179
census_tract_1h + log1p	RBF	0.5558	0.6783
census_tract_1d (no TE)	Poly	0.3060	0.9668
census_tract_4h (no TE)	Poly	0.2904	2.2567
census_tract_1h + log1p (no TE)	RBF	0.2362	1.1639
census_tract_1h (no TE)	Poly	0.2205	1.5755
census_tract_4h + log1p (no TE)	RBF	0.2161	0.8173
census_tract_1d + log1p (no TE)	Poly	-249.6207	1.2963

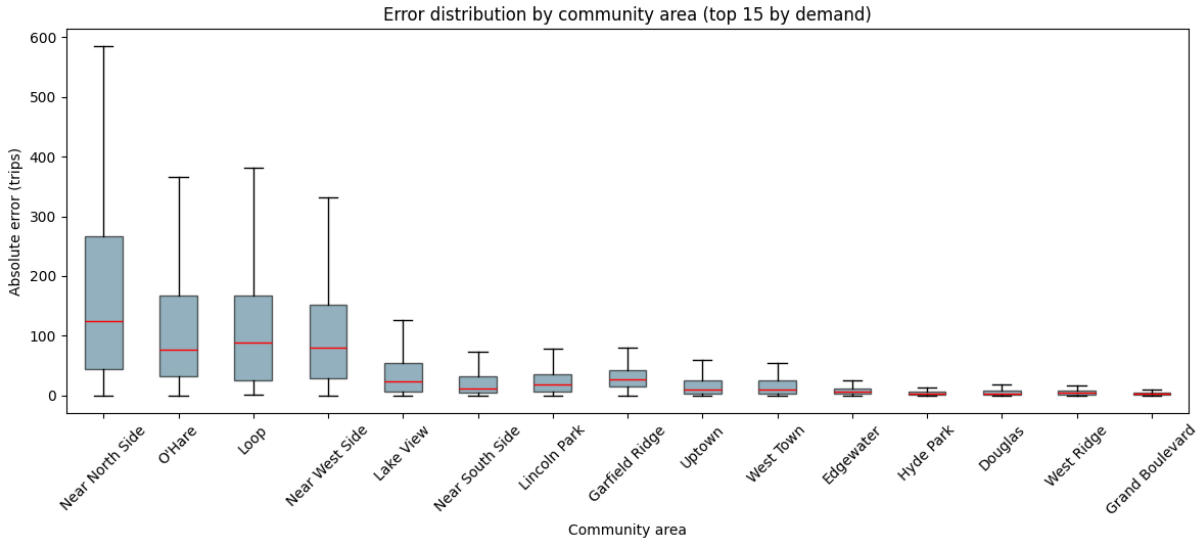

Figure 3: Absolute error distribution by community area (top 15 by demand) for the recommended configuration (community area, 4h, log1p, RBF).

Table 4: SVR prediction accuracy under absolute and relative tolerances

experiment str	n_test i64	exact_match f64	within_1_trips f64	within_2_trips f64	within_ f64
”census_tract_1h”	1449082	71.5	94.9	96.6	98.2
”census_tract_1h + log1p”	1449082	95.3	96.3	97.0	98.1
”census_tract_1h + log1p (no TE…	1449082	86.0	96.4	97.1	98.1
”census_tract_1h (no TE)”	1449082	58.5	92.7	96.1	98.0
”census_tract_4h”	362341	26.7	91.0	93.6	96.0
”census_tract_4h + log1p”	362341	92.3	93.6	94.5	96.0
”census_tract_4h + log1p (no TE…	362341	92.7	93.8	94.7	96.0
”census_tract_1d + log1p (no TE…	60449	72.3	88.9	90.4	92.1
”census_tract_1d”	60449	28.9	80.6	87.4	92.0
”community_area_1h + log1p”	235897	46.3	76.6	85.5	92.0
”community_area_1h + log1p (no …	235897	45.0	75.8	85.2	91.9
”census_tract_1d + log1p”	60449	86.1	88.0	89.6	91.8
”community_area_1h”	235897	30.4	68.5	82.1	91.4
”community_area_4h + log1p”	58985	22.3	50.1	65.1	82.2
”community_area_4h + log1p (no …	58985	22.2	49.7	64.4	81.4
”community_area_1h (no TE)”	235897	8.7	25.7	41.1	74.2
”census_tract_4h (no TE)”	362341	6.7	19.9	33.1	68.1
”community_area_4h”	58985	6.1	18.0	29.7	59.8
”census_tract_1d (no TE)”	60449	5.7	16.3	26.2	55.6
”community_area_4h (no TE)”	58985	5.5	15.9	25.9	51.5
”community_area_1d + log1p (no …	9840	6.1	17.5	27.0	47.8
”community_area_1d + log1p”	9840	4.9	13.5	21.5	40.5
”community_area_1d”	9840	4.5	12.6	20.7	39.4
”community_area_1d (no TE)”	9840	2.2	6.7	11.2	24.9

7.2 Neural Network Results

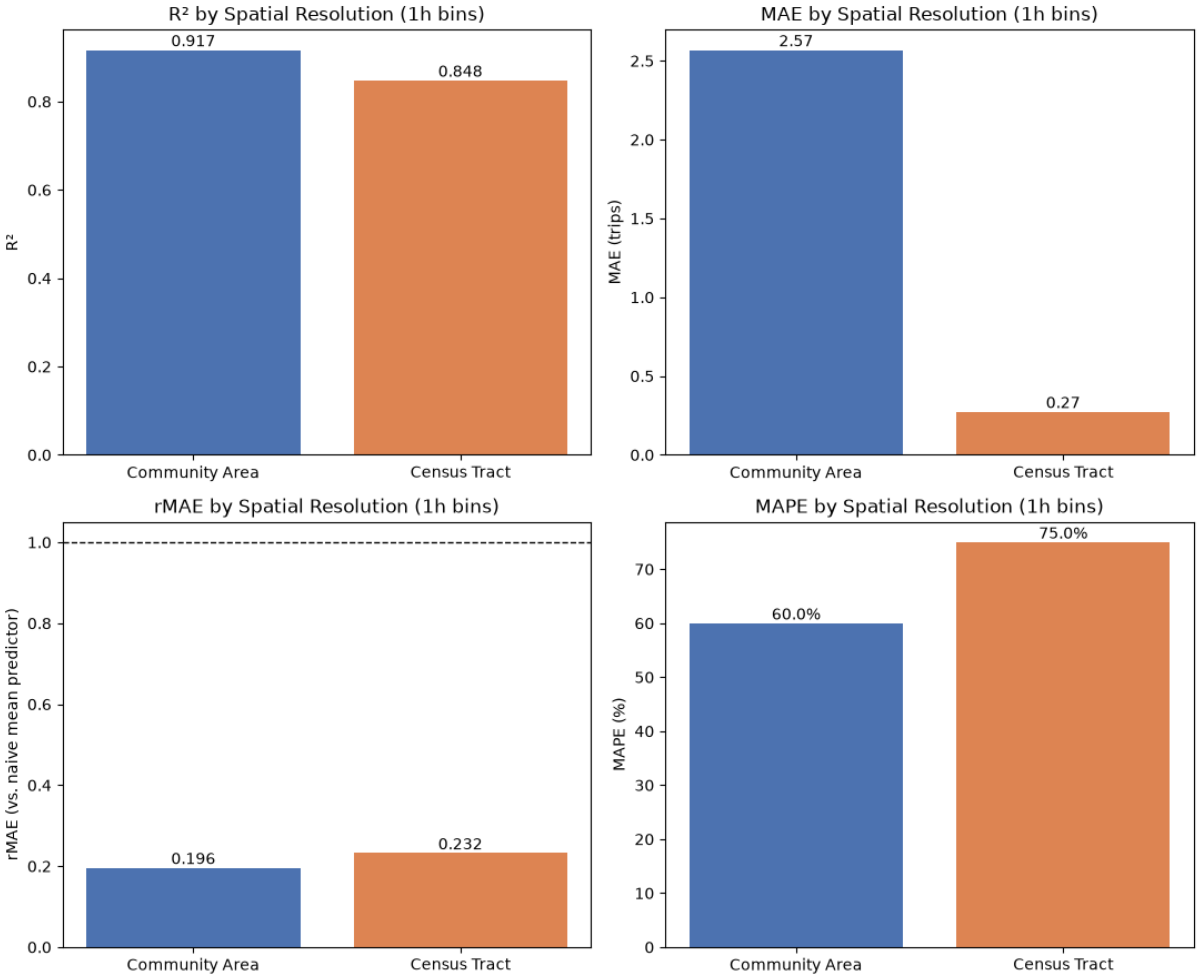

Figure 4: Neural network performance by spatial resolution (1h bins).

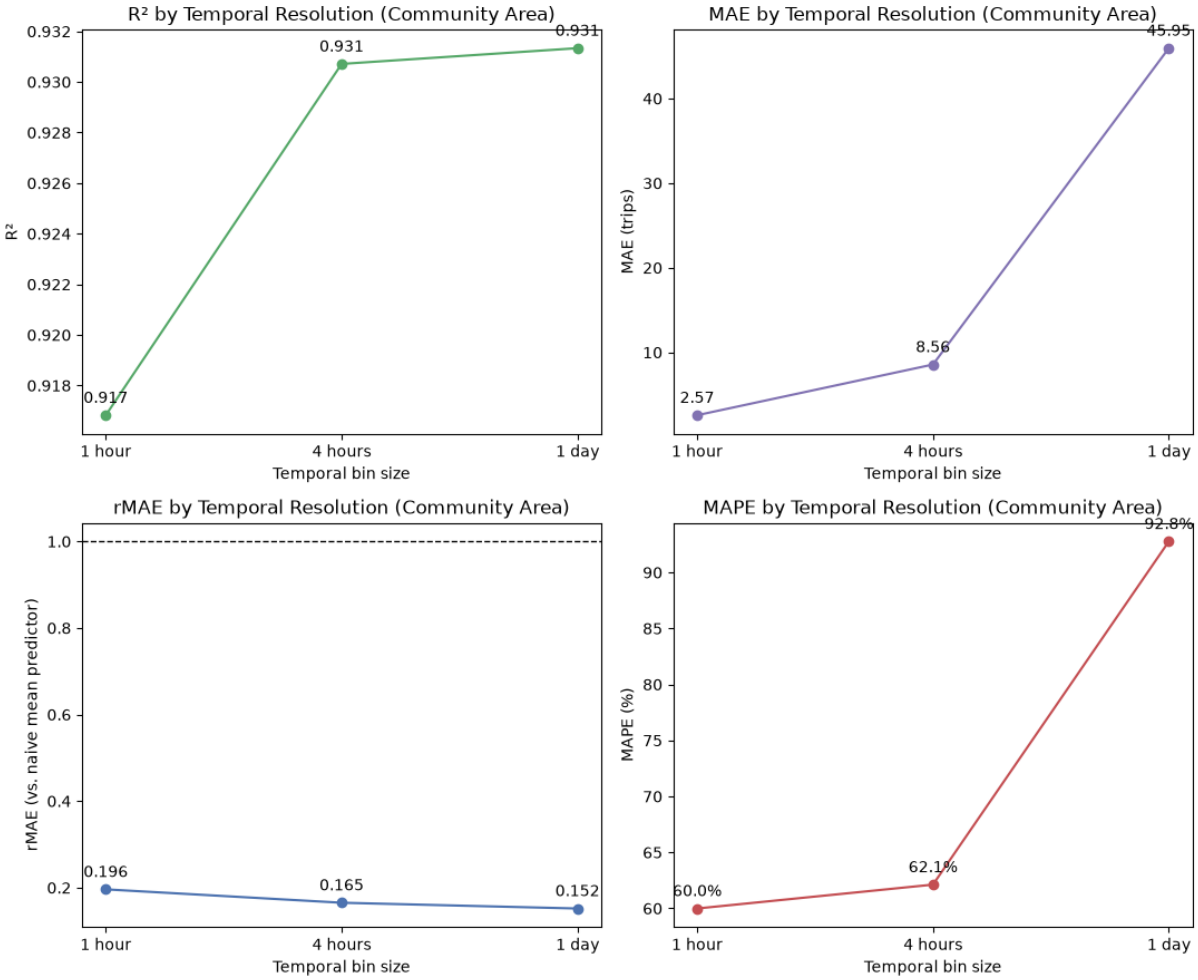

Figure 5: Neural network performance by temporal resolution (community areas).

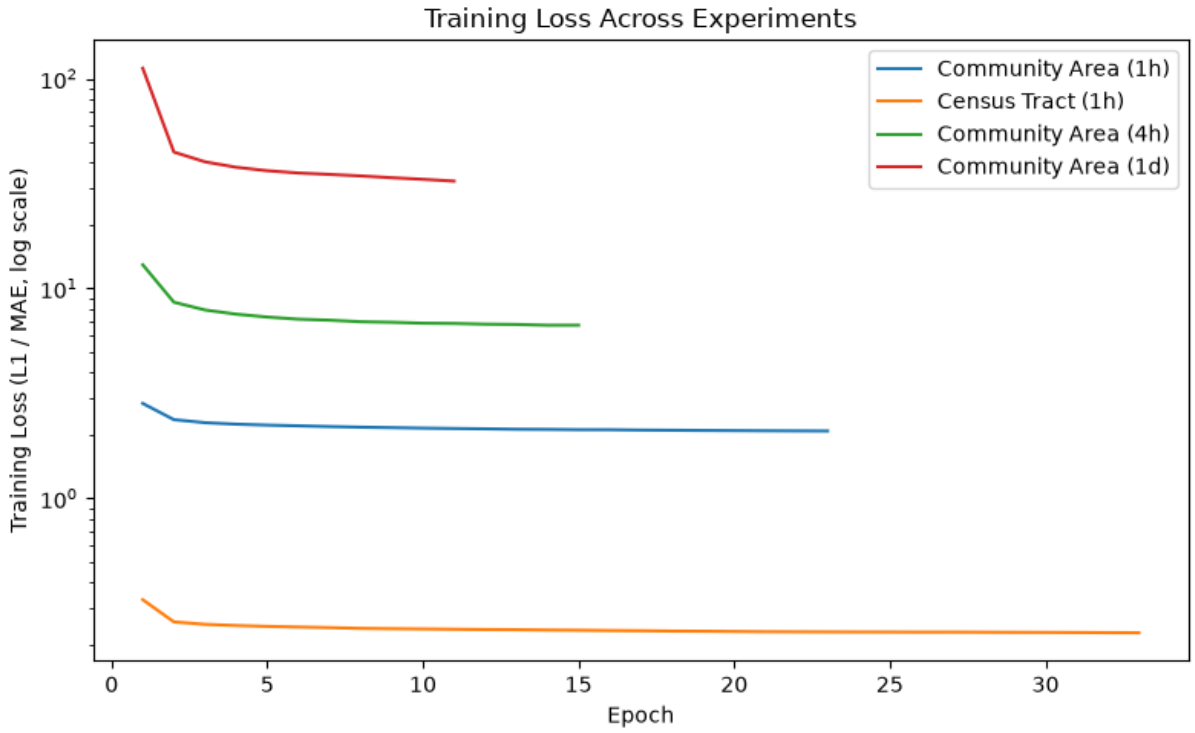

Figure 6: Training loss curves across all four experiments.

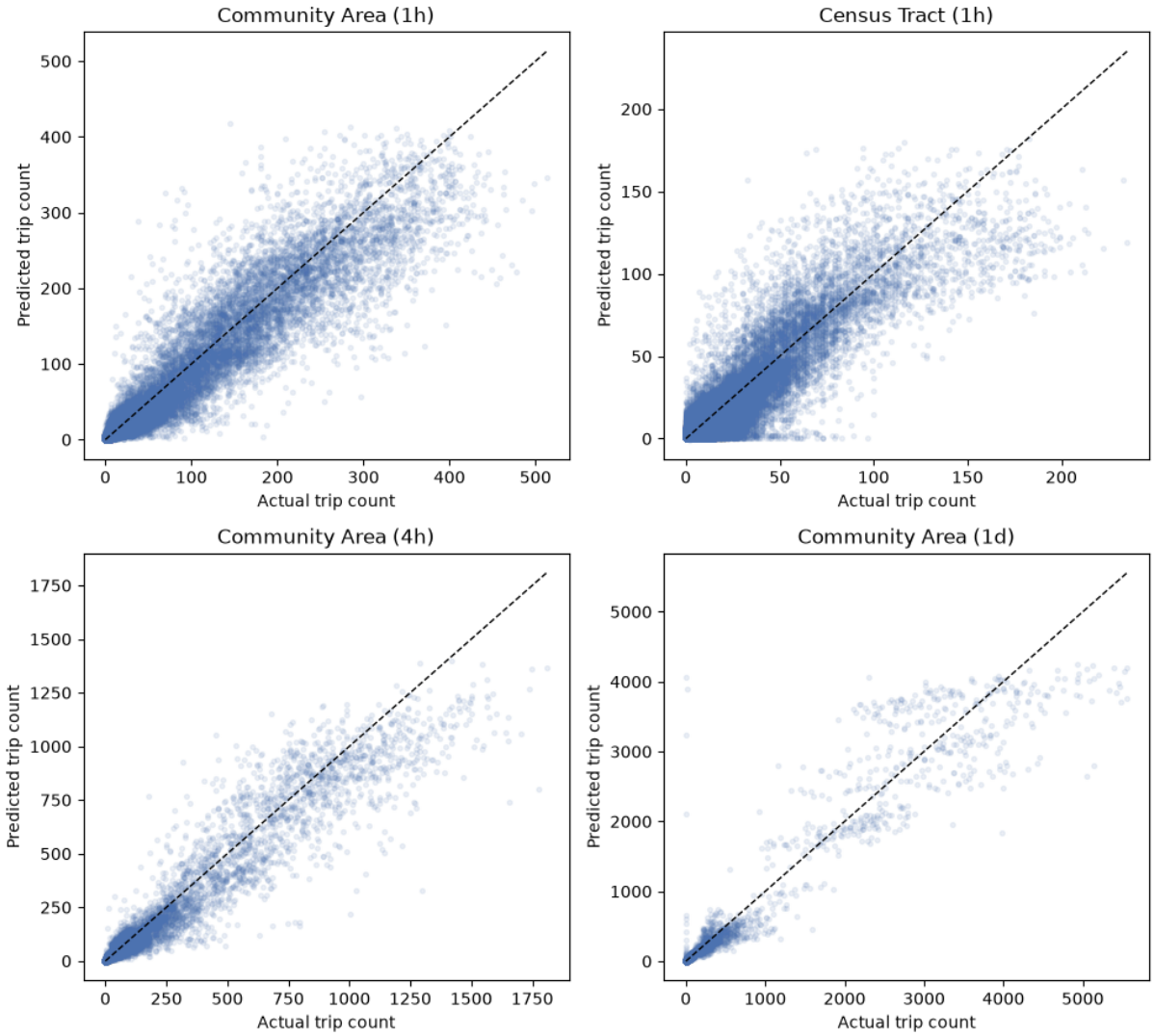

Figure 7: Predicted versus actual trip counts on the test set.

Table 5: Neural network prediction accuracy under absolute and relative tolerances

Table 5

	experiment	n_test	exact_match	within_1_trips	within_2_trips	within_5_trips	w
0	Community Area (1h)	235897	47.6%	76.2%	85.9%	92.8%	1
1	Census Tract (1h)	1449082	95.7%	96.5%	97.2%	98.4%	7
2	Community Area (4h)	58985	25.8%	52.2%	66.3%	82.8%	1
3	Community Area (1d)	9840	4.8%	14.4%	23.3%	43.7%	1

References

Team, Q. (2024). Quarto: An open-source scientific and technical publishing system. *Journal of Modern Data Science*. https://quarto.org

Wickham, H., Çetinkaya-Rundel, M., & Grolemund, G. (2023). *R for data science*. O'Reilly Media.